Kayal Holidays

Tour operator website, package CMS and AI-assisted enquiry inbox. Technical Design Document.

GitHub	https://github.com/23f3000361/kayal-holidays
Live app	Not deployed at the time of writing. The app runs locally in about two minutes (see `deployment.md`); the Vercel + Turso set-up is documented and needs only account credentials.
Brief	1. Tour operator's website with custom CMS for packages, blogs and enquiries (Company A, Kochi)
Core slice	Public package pages · package CMS with seasonal prices and preview · enquiry form · admin enquiry inbox · AI enquiry extraction and package suggestions
Date	3 November 2026

1. Summary

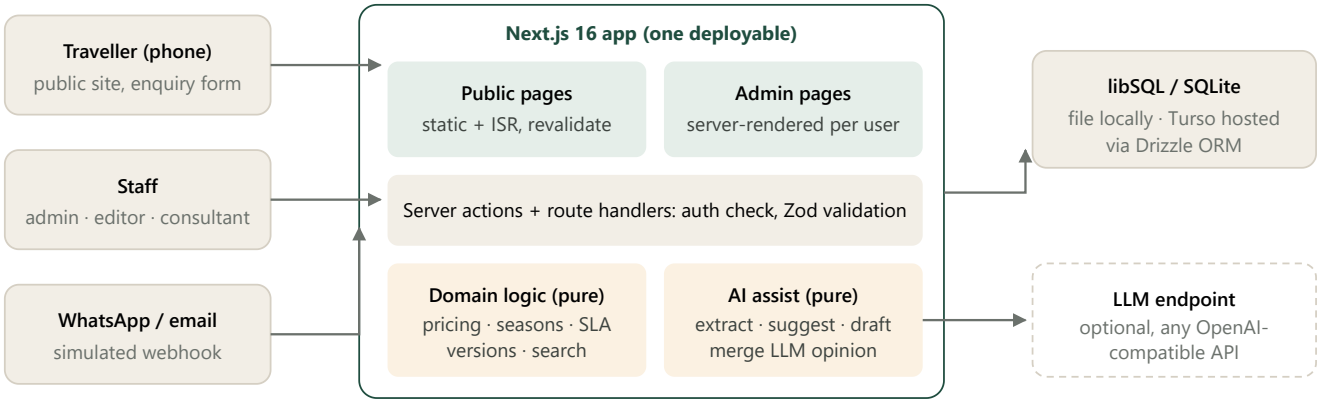
The client loses an estimated 48 bookings a month (₹4.37 lakh in margin) because a quarter of web enquiries wait more than 12 hours for a reply, and pays ₹3,000 and 3 days for every package or price change. This build attacks both:

- **Self-service CMS.** An editor changes a seasonal price, previews it and publishes it, or schedules it, in about two minutes. Every version is kept and can be restored.
- **One inbox with a one-hour clock.** Web, WhatsApp, email, chat and phone enquiries land in one place, get an instant acknowledgement, go to the consultant with the lightest load, and show a live countdown to the one-working-hour target.
- **An assistant that does the slow part.** It reads the message, extracts party, dates, nights, places, budget and must-haves, highlights the words each value came from, ranks three packages with a visible score, and drafts a reply priced from the price table. A consultant checks it and presses Send. Nothing is sent or published automatically.

Beyond the core slice it also ships the rest of the CMS the brief asks for: a **journal (blog) CMS** whose posts carry package cards and related trips, so readers are led to the right package; **team management** for admins; and an **Old URLs** screen where editors import the old site's links as permanent redirects. Plus quotes as a branded web link and PDF, a traveller account, signed preview links for the MD, an activity log, sitemap and structured data, and a measured page load under 2.5 s on throttled 4G.

> **Honest scope note.** The core slice was built and tested first; the blog CMS, team screen and redirect manager were added after it was complete. Messages and payments are simulated, as the brief allows. Hindi and German are not built (section 9). Every assumption is labelled `A1` … `A21` and listed in section 8.

2. Architecture and why



Decision	Why
Next.js 16 App Router, one deployable	The brief needs server-rendered, fast, crawlable pages *and* a secure admin. One TypeScript codebase does both; server components keep data access on the server, and server actions give typed mutations without a separate API layer. One deployable keeps hosting cheap for a 48-person company.
Static generation + incremental revalidation for public pages	Package and destination pages are generated as HTML and cached at the edge. Publishing calls `revalidatePath`, so a price change is live in seconds; a 60-second revalidation window lets *scheduled* versions go live with no cron job. Measured median LCP is 1.9–2.3 s on throttled slow 4G (section 6).
SQLite dialect via libSQL + Drizzle	The reviewer runs it with zero services (a local file); production uses Turso with the same SQL. Drizzle gives typed queries and plain SQL migrations. The data set (60 packages, 2,400 enquiries a month) is small; Postgres would add operations cost without benefit at this size.
Own session auth	Four roles and simple email + password do not justify a third-party identity provider. Sessions are random 256-bit tokens; only their SHA-256 is stored; cookies are httpOnly, SameSite=Lax, Secure in production; passwords use bcrypt.
Pure domain modules	Pricing, seasons, the reply clock, versioning, search and the AI are plain functions with no database or framework imports, so they are unit-tested directly (84 tests) and reused on server and client. The price shown in the browser calculator, in the reply draft, in the quote and in the PDF all come from one `priceParty()`.
Redirects in the proxy, from the database	The old site's URLs must keep their Google ranking, and the client has 140 of them to load and maintain without a developer. The Next.js proxy checks every request against an in-memory copy of the redirect table (refreshed every 30 seconds) and answers with a 308, counting hits. If the table is unreachable it simply lets the request through.
Rules engine first, LLM optional	See section 4. It runs free, offline and deterministically; personal data never leaves the system unless the client opts in to an LLM.

3. Data model

Table	Key fields	Notes
users	email, name, password_hash, role (admin | editor | consultant | traveller), active, last_assigned_at	Staff accounts are seeded or created by an admin; public sign-up always creates travellers.

sessions	id = sha256(token), user_id, expires_at	Raw token only in the cookie.
packages	code (PKG-KL-07), slug, archived	Stable identity; the slug is locked after first publish so URLs never break.
package_versions	package_id, kind (draft | release), version_no, content JSON, visibility (public | hidden), publish_at, note, author_id	One editable *draft* per package; every publish adds an immutable *release*. Content is validated by a Zod schema.
media	bytes, mime, width, height	Uploads, resized to WebP in the browser, served with year-long cache headers.
enquiries	ref (ENQ-10015), channel, package_id, traveller_id, contact, message, form fields, consent_contact, consent_whatsapp, status, assigned_to_id, extraction JSON, confirmed JSON, first_reply_at, due_at	`extraction` keeps what the AI read; `confirmed` keeps consultant corrections on top, so both stay auditable.
enquiry_events	type (received, ack, ai, note, reply, status, assign, quote, correction), body, actor	The enquiry timeline.
messages	channel (email | whatsapp), to, subject, body, kind (ack | reply)	The simulated outbox; nothing leaves the system.
quotes	ref, token, package + *version* id, travel date, party, price JSON, valid_until	Pinned to the exact package version quoted, so later edits never change a sent quote.
blog_posts	slug, draft JSON, published JSON, publish_at, author_id	Saving edits the draft; publishing copies it across, so a live post never shows half-finished edits. A post never yet live can be scheduled.
redirects	from_path (normalised), to_path, hits, last_hit_at	Old-site URLs; hit counts show which old links still bring traffic.
audit_log, **counters**	actor, action, summary · name, value	Activity log; atomic reference numbers.

Versioning, the hard part

The live version is computed, not stored: *among releases whose publish time has passed, the highest version number wins; if it is hidden, the package is offline.* Consequences, all unit-tested:

- **Scheduling needs no job runner.** A release scheduled for 1 October simply becomes the answer on 1 October, and the 60-second page revalidation picks it up.
- **Unpublish is a release** (visibility hidden), so the history shows who took a package offline and when.
- **A newer publish always wins.** Testing exposed a trap: if v3 is scheduled for March and an editor publishes an urgent v4 today, a "latest publish time wins" rule would let v3 silently undo v4 in March. The rule above prevents that, and publishing openly cancels the superseded schedule. The publish dialog warns first and shows a field-by-field diff against the live version.
- **Preview** renders the draft with the very component the live page uses, so what you preview is what goes live. A signed, 7-day link lets the MD review on a phone without an account.

4. How the AI part works

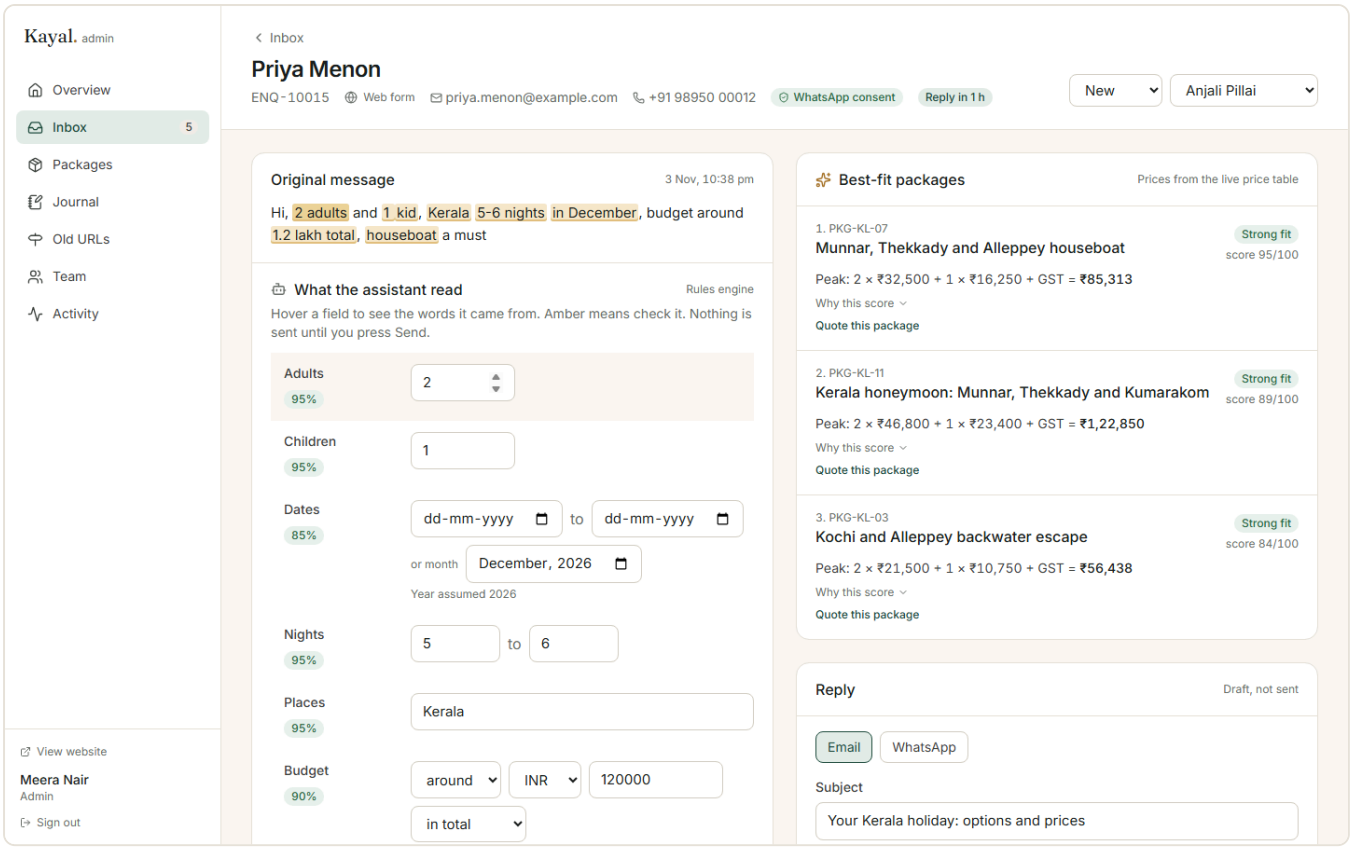
The brief asks the AI to extract dates, travellers, destination and budget, suggest three packages and draft a reply, with corrections beside the original, nothing sent without a person, and prices never from the AI. The pipeline:

1. **Extraction (rules engine, always on).** Pattern rules and a gazetteer of the client's destinations (including spellings travellers really type, such as Alappuzha, Cochin or Kodai) read: adults and children (and their ages), dates (ranges, months, "next month", holidays, Indian day/month order), nights ("5-6 nights", "6D/5N", "a week"), places and states, budget (lakh, k, ₹, $, €, £; total versus per person; around versus up to), must-haves versus nice-to-haves ("houseboat a must"), trip type and hotel class. Each field carries a **confidence** and the **character spans** of its evidence. Unclear cases are scored low instead of guessed: a group size gives adults at 60% ("assumed all adults"); "budget 80k" without total or per person gives 65%; "05/01" is read as 5 January at 55%; non-English text raises a warning.
2. **LLM second opinion (optional).** If `LLM_*` variables are set, any OpenAI-compatible model is asked for the same fields as JSON, validated by Zod, and merged field by field: if they agree, confidence rises; if they disagree, the field is flagged with both readings; a value found only by the model is kept at reduced confidence, and lower still if it cannot be found in the message text, which guards against invented details. Timeouts and errors fall back silently to the rules result.
3. **Suggestions.** Only *live* packages are ranked (drafts like PKG-SI-12 never appear). The score is a visible sum: destination 35, must-haves or interests 25, length 20, budget 20 (priced for the actual party and date), trip style 5. A missing must-have halves the score. Each suggestion shows "why this score".
4. **Draft reply.** Plain-text template filled from the fields and the price table. It asks only about what is missing or uncertain (child ages, exact dates). Every rupee figure comes from `priceParty()`; a unit test checks that every ₹ amount in a draft matches a price-table calculation.

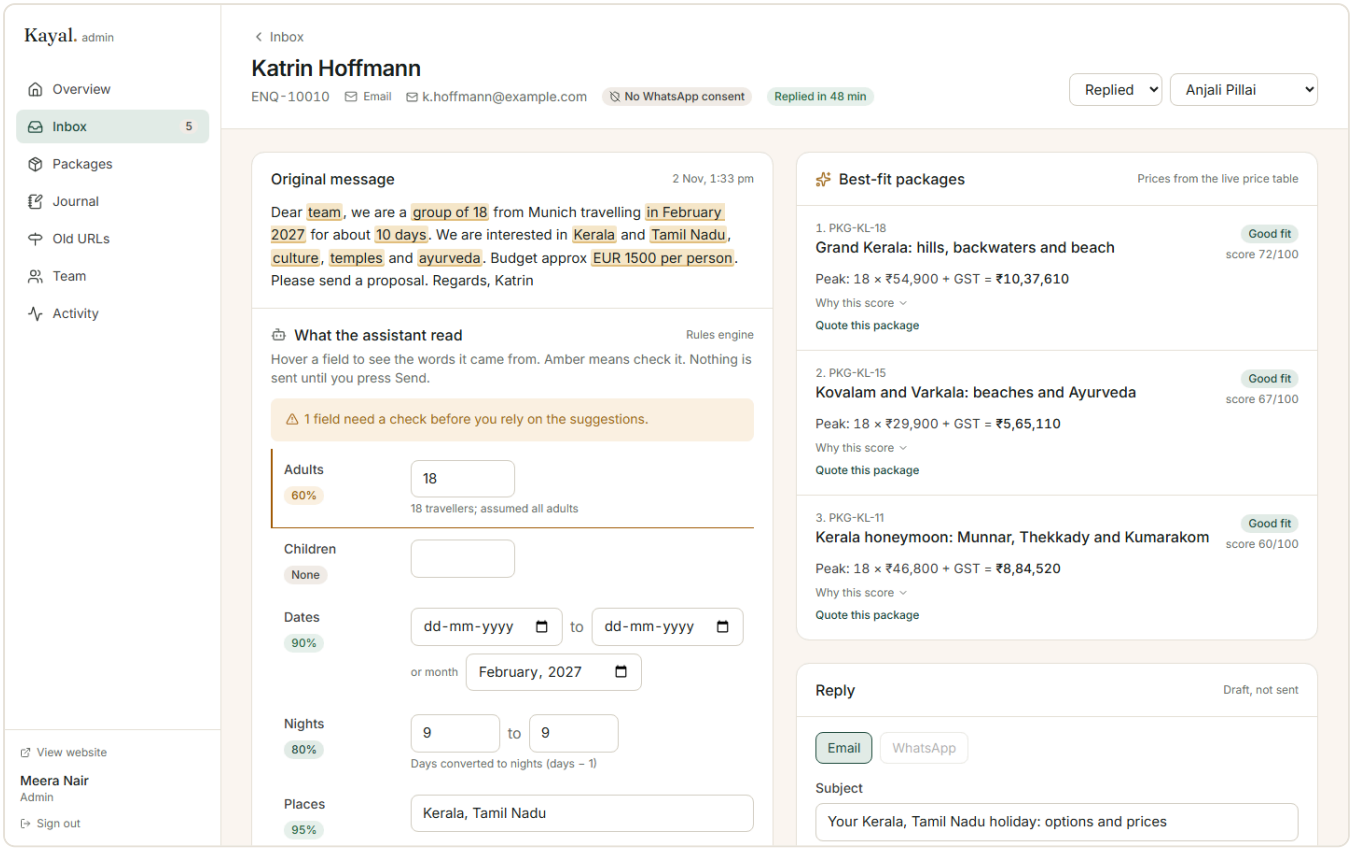
When it is wrong or unsure

- The original message sits beside the fields with evidence highlighted; hovering a field lights up its words. Fields below 70% confidence, and required fields not found, are marked amber.
- Consultants correct fields and confirm; suggestions and the draft re-run on the corrected values; corrections are stored separately and logged on the timeline.
- Nothing is sent without a person: Send needs a second confirmation. WhatsApp is disabled unless consent is on record.
- Structured values the traveller typed in the form (date, adults, children) outrank anything inferred.
- Quotes can only be created for live packages and dates that have a price; otherwise the consultant is told to choose another date or package.

Sample from the brief. "2 adults and 1 kid, Kerala 5-6 nights in December, budget around 1.2 lakh total, houseboat a must" extracts every field correctly (budget at 90% confidence), ranks PKG-KL-07 first (score 95), and prices it for 2 adults + 1 child in peak season at ₹81,250 + 5% GST ₹4,063 = **₹85,313**, within the budget. The draft asks the child's age and exact dates. All of this is asserted in tests.



Enquiry workspace for the brief's sample enquiry: evidence highlighted in the original, editable fields with confidence, ranked packages and a priced draft.



Uncertainty shown, not hidden: a group of 18 is read as 18 adults at 60% and flagged amber; the foreign-currency budget notes the assumed exchange rate.

5. Main flows

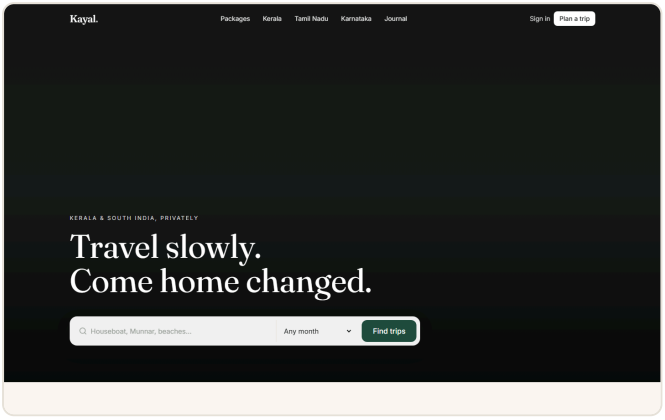
Flow	Steps
Traveller enquires	Package page → Enquire (the one primary button; sticky on phones) → form with consent and optional WhatsApp consent → server validates (Zod, honeypot, 5 per 10 min per IP) → enquiry stored, extraction run, consultant assigned, acknowledgement sent from a fixed template, reply due time set → thank-you page with reference and expected reply time.
Other channels	Staff "Log an enquiry" for calls and pasted chats; `/api/inbound/whatsapp|email|chat` webhook (shared secret) for simulated providers. Same pipeline as the web.
Consultant replies	Inbox (Open / Mine / Unassigned / Closed, search, channel filter, live timers) → enquiry → check amber fields → confirm → pick from suggestions → edit draft → Send (email, or WhatsApp if consented) → first-reply time recorded, status Replied. Optional quote: package + date + party → priced from the table → link + PDF.
Editor changes a price	Packages → package → Prices (season cards with gap and overlap checks) → Save draft (Ctrl+S) → Preview → Publish… (diff against live, warnings, now or scheduled, change note) → public page revalidated.
New package	New package (code, title, region) → editor checklist lists what is missing (summary, photos, prices, itinerary length…) → publish when the list is clear. Or Duplicate an existing package.
Editor writes a post	Journal → New post → write with the formatting toolbar and live preview → place a package card in the text where readers should act → "Suggest from the text" proposes related packages (the inbox's matching engine run on the prose; the editor chooses) → checklist (cover, excerpt, length, at least one related package) → Publish now or schedule. Each package page lists the posts that link to it ("Read before you go").
Old URLs	Old URLs → paste the crawl as two columns → Check list shows, row by row, what will be imported and what is skipped and why (dead target, chain, duplicate, or an old URL that would hide a live page) → Import. Targets that later disappear (a package unpublished) are flagged red.
Team	Team → add a person with a role; a one-time password is shown once to pass on privately. Change roles, reset passwords, deactivate. Nobody can deactivate or demote themselves and one active admin always remains; deactivation signs the person out at once and returns their open enquiries to the unassigned pool.
Traveller account	Register → enquiries sent while signed in appear in "Your trips" with replies and quotes. Enquiries are *not* linked by email address, so nobody can see another person's enquiries by registering their email A16.

Roles and access. One permission table (`lib/permissions.ts`) is checked on the server in every page, action and route. Admins see everything and manage the team. Editors manage packages, the journal and old URLs, but cannot see enquiries (personal data is minimised). Consultants see their own enquiries and the unassigned pool, and can take an unassigned one; only admins reassign. Travellers see only their own trips. The proxy redirect for signed-out users is a convenience; it is not the security boundary.

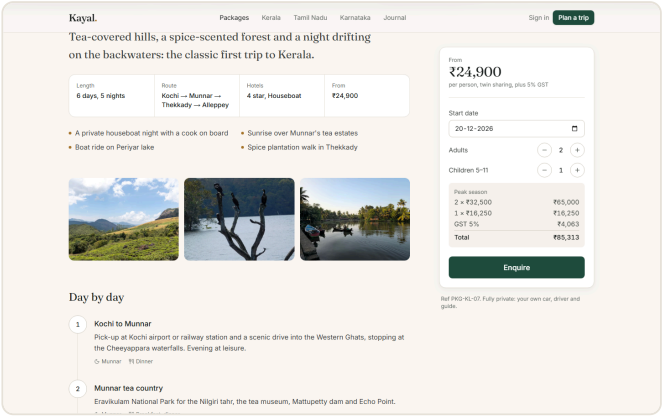
6. UI and design decisions

The client asked for restraint: white space, a restrained palette, clear type, and nothing without a purpose. Travel is sold by feeling. The system is:

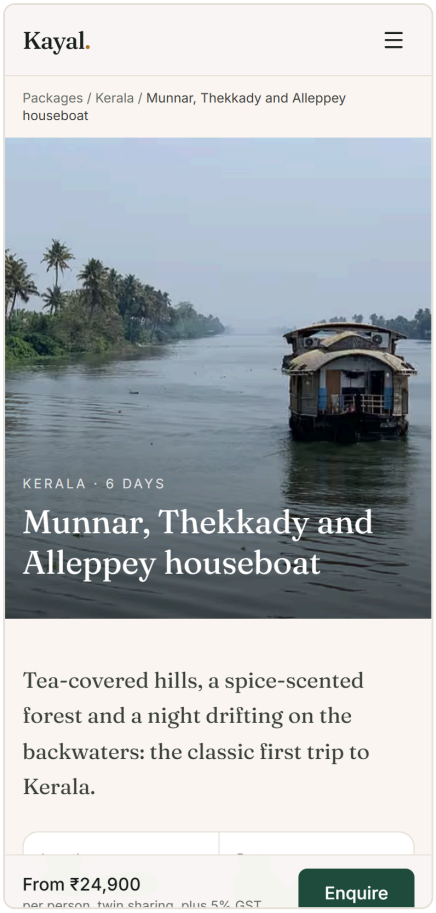
- **Palette:** warm paper (#FAF8F4), ink (#1D211F) and **one** backwater green (#1F4D3F) reserved for actions, so the Enquire button is always the obvious next step. Gold appears only as a small accent. Amber and red mean “check this” and “late”, and nothing else.
- **Type:** Fraunces (serif) for headings on the public site gives quiet editorial warmth; Inter for everything else. The admin uses Inter only: calm and dense.
- **Photography first:** full-bleed covers with a gentle gradient for legible titles; a horizontal swipe gallery on phones.
- **Prices always state their basis:** “per person, twin sharing, plus 5% GST” sits next to every price, and the season table shows uncovered dates as “On request” instead of hiding them.
- **Admin for non-technical editors:** plain-language labels (“Day by day”, “What changed (for the history)”, “Old URLs” rather than “redirects”), a live “Ready to publish” checklist, one Save and one Publish, and no jargon such as “slug”. The journal editor is Markdown underneath, but a toolbar and a live preview mean nobody has to learn it.
- **States:** skeleton loading screens, empty states with a next step, inline validation messages, friendly error and 404 pages, and disabled buttons with spinners while saving.
- **Phones:** every screen was checked at 390 px; the public package page has a sticky price-and-Enquire bar; the admin navigation becomes a horizontal tab strip.
- **Accessibility:** semantic headings and landmarks, a skip link, labelled controls (the e2e tests select by accessible name), visible focus, reduced-motion support, and live regions for prices and timers.



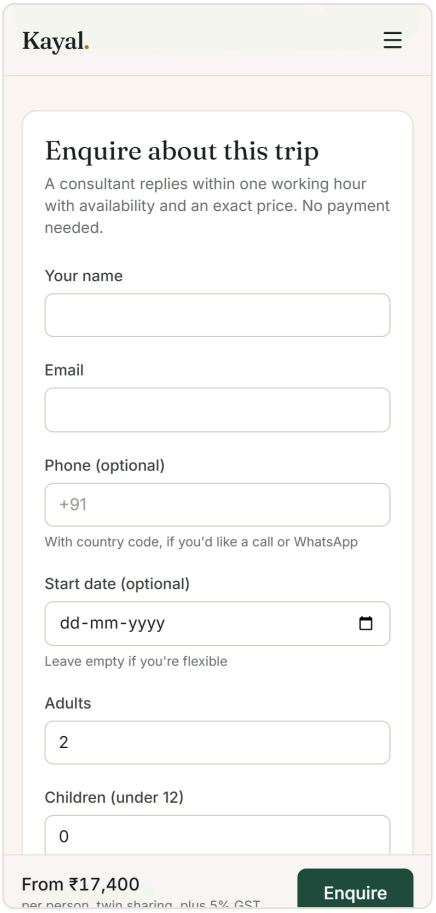
Home: one image, one sentence, one search.



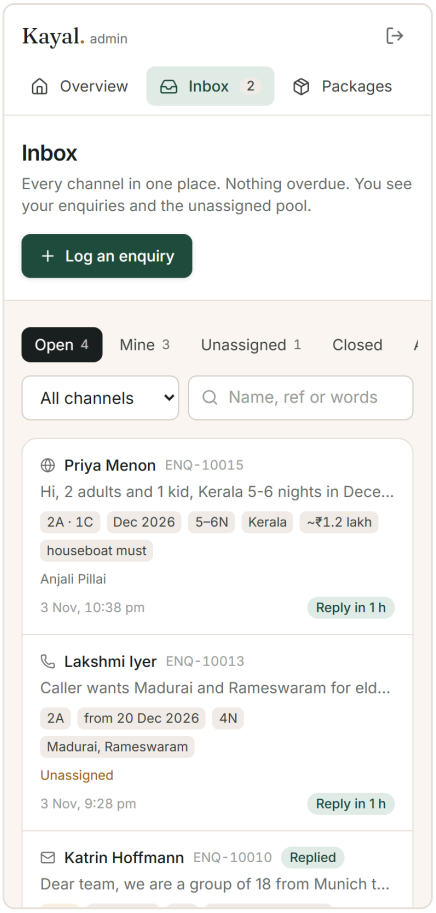
Package page: summary, facts, and a sticky price card with a live calculator.



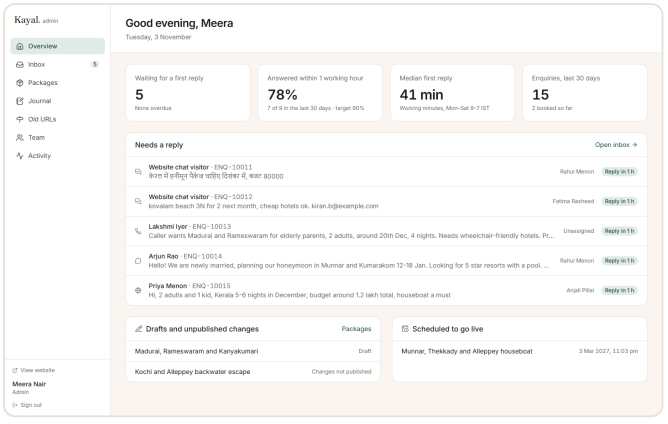
Package on a phone, sticky Enquire bar.



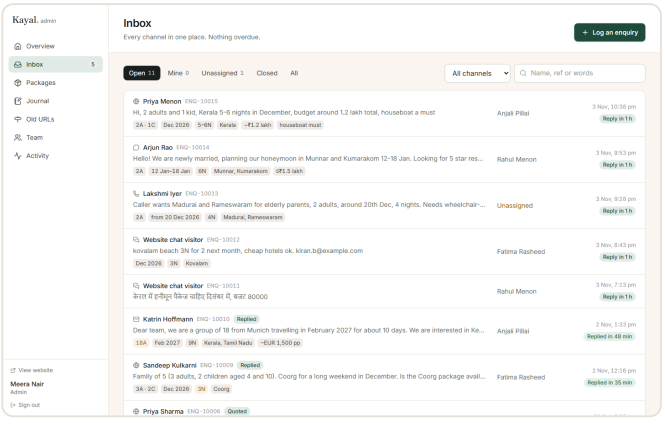
Enquiry form on a phone.



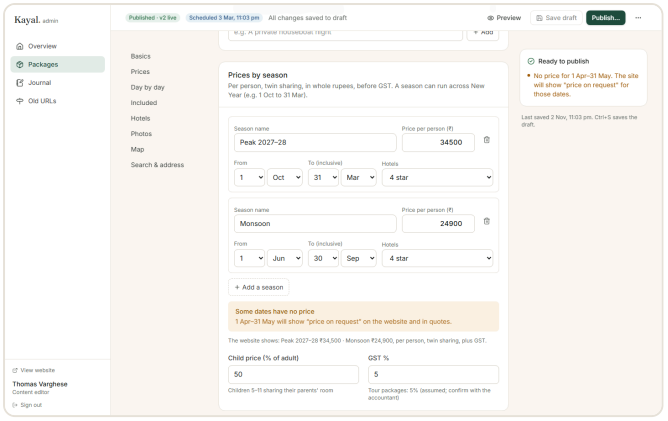
Consultant inbox on a phone.



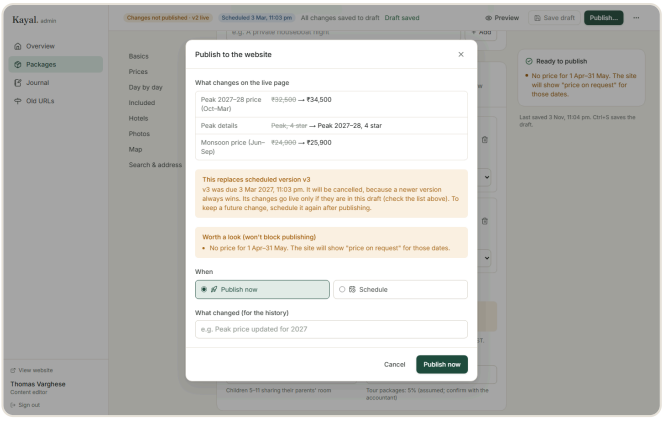
Overview: the client's success metric (answered within one working hour, target 90%) is the first number.



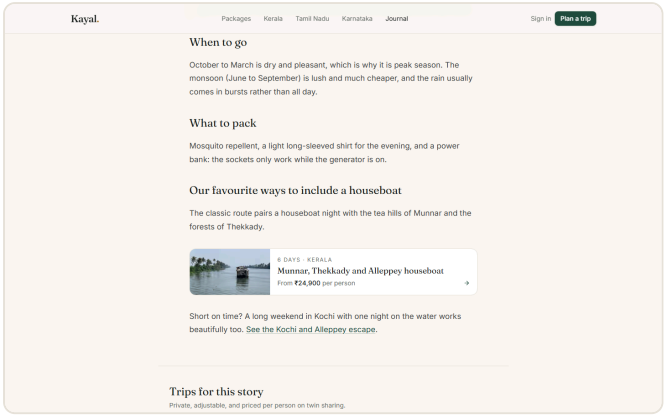
Inbox: channels, extracted chips (amber = uncertain), owner and live reply timer.



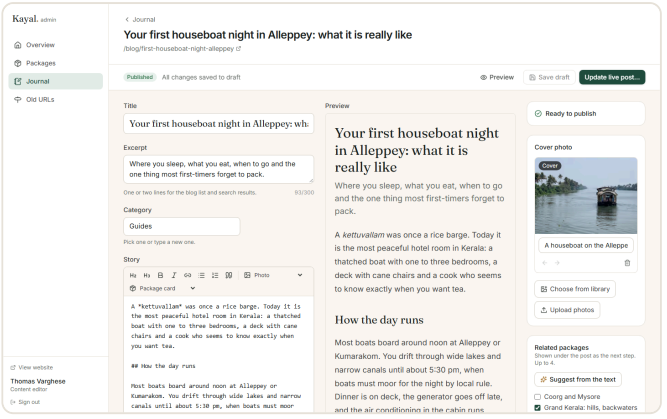
Editor: seasonal prices with automatic gap and overlap checks.



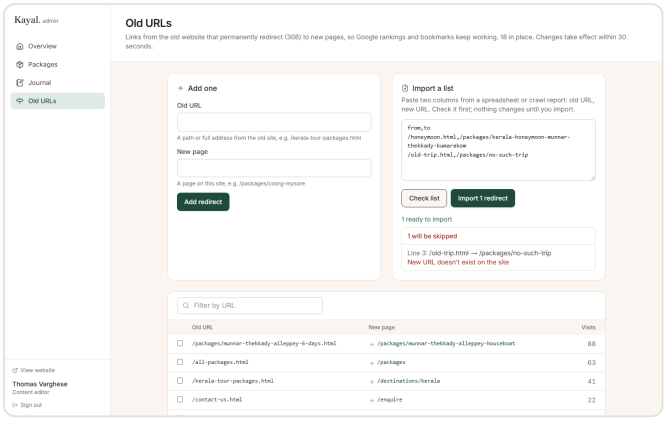
Publish dialog: diff against live, warnings, and now or scheduled.



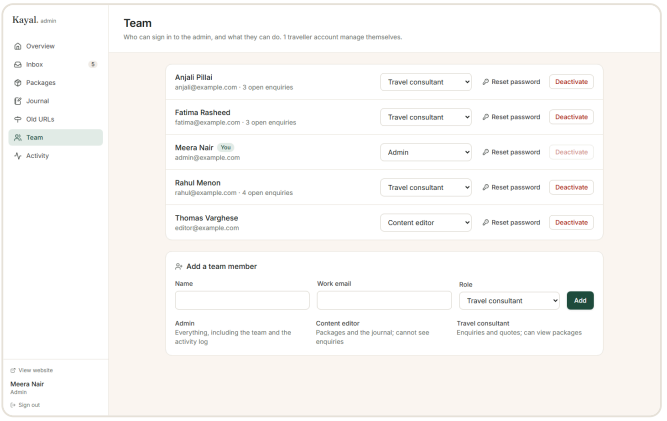
Journal post: a package card in the story, then "Trips for this story".



Journal editor: toolbar, live preview, cover, related packages with suggestions.



Old URLs: a pasted list is checked row by row before anything changes.



Team: roles in plain words; self-lockout is impossible.

Performance, measured

`scripts/perf.mjs` loads pages in Chromium as a Pixel 7 with Lighthouse's "slow 4G" throttling (150 ms RTT, 1.6 Mbps) and a 4× CPU slowdown; median of three runs, production build:

Page	Median LCP	Transferred
Home	2.30 s	538 KB

All packages	1.93 s	624 KB
Package page	2.16 s	547 KB
Destination (Kerala)	2.22 s	547 KB

Measuring led to two fixes that together took the package listing from 3.3 s to 2.0 s: the listing became a cached page with in-browser filtering, and card photos are now served at quality 60. A third finding: an entrance fade on the home headline kept the text invisible for over 1.5 s on slow loads, so it was removed. Leaflet maps load only when scrolled into view. Figures are measured with the redirect check in the proxy on every request. They come from a local server, so first-byte time on a CDN will differ, and single runs vary by a few hundred milliseconds.

7. Security, privacy and quality

- **Secrets:** none in the repository; `.env.example` lists names only; production refuses to start preview signing without `APP_SECRET`.
- **Content safety:** journal posts use a small Markdown subset parsed into a tree, never raw HTML, so a post cannot inject scripts; links are limited to http(s), mailto and same-site paths; images must come from the photo library.
- **Input validation:** Zod on every action and route; IDs checked as UUIDs; uploads checked by magic number and size; the inbound webhook uses a timing-safe secret comparison.
- **Access control:** server-side permission checks plus per-record visibility for enquiries (tested end to end: an editor is refused the inbox; a consultant cannot open a colleague's enquiry).
- **Abuse:** honeypot and per-IP limit on the enquiry form; login lock-out after 5 failures per email + IP for 15 minutes; a dummy password check for unknown emails, so response time does not reveal which accounts exist.
- **Headers:** nosniff, frame options, referrer policy, permissions policy; admin, account, preview and quote pages are noindex.
- **DPDP Act 2023:** purpose-specific consent on the form; separate, recorded WhatsApp consent enforced at send time; editors cannot see personal data; a plain privacy notice; fictional demo data only (example.com addresses).
- **Tests:** 107 unit tests (pricing and GST rounding, season wrap-around and gaps, extraction on 24 phrasings, LLM merge, suggestions, draft price integrity, working-hours SLA, assignment, versioning, search, Markdown safety, related-post suggestions, team lock-out rules, redirect validation) and 21 Playwright runs on desktop and phone (search, prices, draft 404, redirects including case and trailing slash, validation, the sample enquiry end to end, quote PDF, access control, publish updating the live page, publish blocked by a checklist error, writing and publishing a post, importing old URLs, adding and deactivating staff).

8. Assumptions (labelled)

#	Assumption
A1	Brand name "Kayal Holidays" (kayal: backwater in Malayalam) stands in for Company A.
A2	PKG-KL-07 and PKG-SI-12 are as given (including PKG-SI-12 as a draft and KL-07 having no April–May price, shown as "On request"). Their itineraries, hotels and inclusions, and the other 8 packages, are invented sample data.
A3	GST on tour packages is 5% of the package price, editable per package; confirm with the client's accountant.

A4	Children 5–11 sharing their parents' room pay 50% of the adult price (editable per package); under-5s are free. No single supplement in this build.
A5	The season is decided by the trip's start date.
A6	Working hours are Monday–Saturday, 09:00–19:00 IST, with no holiday calendar; the one-hour target counts working minutes only.
A7	Auto-assignment goes to the active consultant with the fewest open enquiries; ties go to whoever was assigned least recently.
A8	Foreign budgets convert at fixed assumed rates (USD 84, EUR 91, GBP 107), and the UI says so.
A9	Without a date, suggestions price on the 15th of the requested month; without a party, 2 adults.
A10	A WhatsApp message to the business counts as consent to reply on WhatsApp in that conversation; web and phone enquiries need explicit consent.
A11	The 140 old URLs are not in the brief. The Old URLs screen imports them from a crawl; 18 representative mappings (with invented visit counts) are seeded to show it working.
A12	Quotes are valid for 14 days.
A13	Enquiries that do not lead to a booking are deleted after 12 months (stated in the privacy notice; deletion job not built).
A14	Package codes follow PKG-<state>-<nn>; the web address is set from the title and locked after first publish.
A15	Consultants see their own and unassigned enquiries; admins see all; editors see none.
A16	Traveller accounts show only enquiries sent while signed in (no email verification in this build).
A17	Photos are openly licensed Wikimedia Commons images, credited at `/credits`; the client would replace them with its own.
A18	Email, WhatsApp and payments are simulated, as the brief allows: messages go to an outbox shown in the app.
A19	The three journal posts are invented sample content.
A20	New staff get a one-time password shown once to the admin, to pass on privately; there is no email invitation in this build.
A21	Editors may manage old URLs (an SEO task); only admins manage the team.

9. What I cut, and what I would do next

Cut, deliberately

- **Real delivery** of email and WhatsApp, and payments (simulated by design).
- **Hindi and German.** Text is English-only and not yet in message files; extracting strings is the first step.
- **A word-processor-style editor** for the journal. The toolbar plus live preview covers the need without the security cost of storing HTML.
- **Journal version history.** Posts keep one draft and one published copy; packages, where prices make history matter, keep every version.

Known limits

- Scheduled releases go live within about 60 seconds of their time (page revalidation window), not to the second.
- The login and enquiry rate limits are in memory per server; several instances would need a shared store.
- One draft per package: two editors working on the same package at once would overwrite each other (last save wins). The next step is an edit lock or a version check on save.
- Photos live in the database; at scale they belong in object storage behind a CDN.
- Redirect changes take effect within 30 seconds (the proxy's cache). A live journal post can't be re-scheduled; updates to it publish immediately.
- When the admin area streams a not-found page, the HTTP status is 200 (the content is still withheld); a dedicated access-denied route would return 403/404.

Next, in priority order

1. Connect a WhatsApp Business provider and transactional email to the existing webhook and outbox; add delivery status.
2. Measure the AI on real traffic: log consultant corrections per field to get accuracy figures, and tune the rules (or enable the LLM) where corrections cluster.
3. Hindi and German routes with hreflang, starting with the public site and package content.
4. Import the real 140 old URLs through the Old URLs screen; connect Search Console; monitor Core Web Vitals in the field.
5. Edit locking, email invitations for new staff, a holiday calendar for the reply clock, and a retention job for DPDP.

Repository structure: `src/lib` pure domain and AI logic with tests · `src/server` data access and services · `src/app` routes, pages and server actions · `scripts` seed, photos, perf and screenshots · `e2e` Playwright tests · `drizzle` SQL migrations.